Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251218**
Nama Lengkap	**Agatha Sabda Alethea Prabawa**
Minggu ke / Materi	**07 / Percabangan dan Perulangan Kompleks**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Struktur Percabangan Kompleks

Percabangan kompleks adalah struktur kontrol dimana kondisi pemilihan tidak hanya terdiri dari satu tingkat, tapi dapat memiliki banyak alternatif percabangan dan bertingkat. Perintah-perintah yang dikerjakan didalamnya bisa lebih dari satu. Berbeda dengan percabangan biasa yang menggunakan satu blok if-else, percabangan kompleks menggunakan satu blok if di dalam if lainnya (if dalam if / nested if).

Keunggulan utama menggunakan percabangan bertingkat adalah efisiensi eksekusi. Ketika suatu kondisi terpenuhi, maka kondisi lain tidak akan diperiksa, sementara pada percabangan biasa semua kondisi akan diperiksa satu per satu walaupun hasilnya sudah jelas dari awal.

Bentuk-bentuk percabangan kompleks. Terdapat enam bentuk yang umum ada pada percabangan kompleks dalam python. Masing-masing percabangan memiliki struktur dan penggunaan yang berbeda.

1. If dalam if. Pada bentuk ini, kondisi kedua hanya akan diperiksa apabila kondisi pertama bernilai benar. Jika kondisi pertama salah, maka seluruh blok dalam akan diabaikan.

```
if kondisi:
    if kondisi2:
        pass
    else:
        pass
```

2. If-else bertingkat. Pada bentuk ini akan ditambahi cabang else diluar if dalam, hal ini memungkinkan kondisi ditangani secara eksplisit.

```
if kondisi:
    if kondisi2:
        pass
    else:
        pass
else:
    pass
```

3. If sekuensial dengan cabang else. Bentuk ini meletakkan if pertama dan if kedua dalam satu blok if luar, sementara else menangani kondisi ketiga secara terpisah.

```
if kondisi:
    if kondisi2:
        pass
else:
    if kondisi3:
        pass
    else:
        pass
```

4. If-else dalam kedua cabang. Pada cabang if maupun else dari kondisi luar masing-masing memiliki percabangan lebih lanjut. Bentuk ini menghasilkan empat jalur eksekusi yang berbeda.

```
if kondisi:
    pass
    if kondisi2:
        pass
    else:
        pass
else:
    if kondisi3:
        pass
    else:
        pass
pass
```

5. Empat kondisi bertingkat dengan satu output. Pada bentuk ini, empat kondisi akan disusun seacara bertingkat ke dalam. Suatu pernyataan hanya akan dieksekusi ketika keempat kondisi benilai benar. Seperti operator logika and, tapi lebih eksplisit.

```
if kondisi:
    if kondisi2:
        if kondisi3:
            if kondisi4:
                pass
```

6. rantai kondisi else-if. Bentuk ini meletakkan kondisi-kondisi di tiap blok else masing-masing. Seperti bentuk elif, tapi dalam bentuk if bertingkat.

```
if kondisi:
    pass
else:
    if kondisi2:
        pass
    else:
        if kondisi3:
            pass
        else:
            if kondisi4:
                pass
            else:
                pass
```

Perbandingan antara elif dan if-if berurutan sebagai berikut.

1. Pada proses evaluasi:
    - If-elif-else: berhenti di kondisi yang pertama kali terpenuhi, kondisi berikutnya tidak di cek.
    - If-if-if: semua kondisi di cek satu per satu dari atas ke bawah.
2. elif lebih efisien karena jika satu kondisi terpenuhi, maka kondisi lainnya akan dilewati. Sementara itu, if-if kurang efisien karena semua kondisi akan tetap dicek meskipun sudah ada yang terpenuhi.

Struktur Perulangan Kompleks

Perulangan kompleks (nested loops) adalah struktur di mana sebuah perulangan berada dalam perulangan lain. Perulangan bagian luar disebut outer loop, sementara perulangan di bagian dalam disebut sebagai inner loop. Untuk setiap satu iterasi outer loop, inner loop akan berjalan penuh dari awal hingga akhir.

Bentuk ini berguna untuk masalah yang memiliki struktur dua dimensi, seperti segita matriks, catur, berbagai pola bilangan atau teks, dan sebagainya. Pada perulangan kompleks, terdapat dua sintaks yaitu for loop dan while loop.

1. For loop

```
for i in range(n):
    for j in range(m):
        pass
    pass
```

Outer loop = mengulang m kali.

Inner loop = mengulang m kali.

Perintah inner = total dieksekusi -> n*m kali.

Perintah outer = dieksekusi m kali.

2. While loop

```
i = 0
while i < n:
    j = 0
    while j < n:
        pass
        j = j + 1
    pass
    i = i + 1
```

Ada dua perintah dalam perulangan kompleks yang penting untuk diingat. Break dan Continue, merupakan dua perintah yang digunakan untuk menghentikan perulangan.

Perintah break. Perintah ini digunakan untuk menghentikan perulangan yang sedang berjalan sebelum kondisi perulangan selesai. Perintah ini biasa dikombinasikan dengan if untuk menghentikan loop ketika kondisi tertentu terpenuhi. Contoh sebagai berikut.

```
for i in range(20):
    print(i, end=" ")
    if i == 5:
        break

PROBLEMS    OUTPUT    TERMINAL    ···    Pyt

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlP
ata/Local/Microsoft/WindowsApps/python3.11.
 SEMESTER 2/PrakAlPro/testing.py"
0 1 2 3 4 5
```

Perintah continue. Berbeda dengan break yang menghentikan seluruh perulangan, perintah continue hanya melewati satu iterasi saat ini dan melanjutkan ke iterasi berikutnya. Semua pernyataan di bawah continue dalam iterasi tersebut tidak akan dieksekusi.

```
for i in range(10):
    if i == 5:
        continue
    print(i, end="  ")

PROBLEMS    OUTPUT    TERMINAL    ···    Python: testing

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:
ata/Local/Microsoft/WindowsApps/python3.11.exe "d:/
 SEMESTER 2/PrakAlPro/testing.py"
0  1  2  3  4  6  7  8  9
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github: https://github.com/gathasja/71251218_Agatha.git

SOAL 1

PrakAlPro-71251218 > minggu07 > soal01.py > primaTerdekat

```python
def primaTerdekat():
        n = int(input("masukan bilangan n: "))
        if n < 2:
            return "tidak ada bilangan prima kurang dari 2"
        for i in range(n-1, 1, -1):
            prima = True
            for j in range(2, int(i**0.5) + 1):
                if i % j == 0:
                    prima = False
                    break
            if prima:
                return f"bilangan prima terdekat < {n} adalah {i}"
print(primaTerdekat())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:/Users/lenovo/AppData/Loca
ro/PrakAlPro-71251218/minggu07/soal01.py"
masukan bilangan n: 21
bilangan prima terdekat < 21 adalah 19
```

Penjelasan perlangkah soal01 sebagai berikut:

1. def primaTerdekat(): -> untuk mendefinisikan fungsi bernama primaTerdekat().
2. n = int(input("masukan bilangan n: ")) -> untuk meminta user mengisikan bilangan ke variabel n dan mengkonversinya ke integer.
3. if n < 2:

   return "tidak ada bilangan prima kurang dari 2" -> kondisi jika nilai n kurang dari 2 maka fungsi akan mengembalikan pesan bahwa tidak ada bilangan prima kurang dari 2.
4. for i in range(n-1, 1, -1): -> melakukan perulangan dari bilangan terbesar yang kurang dari n (n-1) turun hingga 2.
5. prima = True -> menginisialisasi variabel prima dengan nilai true (sebagai asumsi 'i' adalah bilangan prima).
6. for j in range(2, int(i**0.5) + 1): -> Melakukan perulangan untuk mengecek apakah i habis dibagi oleh bilangan dari 2 sampai akar kuadratnya.
7. if i % j == 0: -> Mengecek apakah i habis dibagi j (tidak bersisa).

8. prima = False -> Jika ditemukan pembagi, maka i bukan bilangan prima. Variabel prima diubah menjadi False.
9. break -> Keluar dari perulangan pengecekan karena sudah ditemukan bahwa i bukan prima, tidak perlu cek lebih lanjut.
10. if prima:

    return f"bilangan prima terdekat < {n} adalah {i}" -> Jika i adalah bilangan prima, fungsi mengembalikan pesan hasil dan berhenti. Karena loop berjalan mundur, i yang ditemukan pertama adalah bilangan prima terbesar yang kurang dari n.
11. print(primaTerdekat()) -> Menjalankan fungsi primaTerdekat() dan mencetak hasilnya.

SOAL 2

PrakAlPro-71251218 > minggu07 > soal02.py > ...

```python
def deret(n):
    for i in range(n, 0, -1):
        faktorial = 1
        for j in range(i, 0, -1):
            faktorial = faktorial * j
        print(f"{faktorial} ", end="")
        for k in range(i, 0, -1):
            print(f"{k}", end=" ")
        print()
n = int(input("n = "))
deret(n)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> pytl
n = 7
5040 7 6 5 4 3 2 1
720 6 5 4 3 2 1
120 5 4 3 2 1
24 4 3 2 1
6 3 2 1
2 2 1
1 1
```

Penjelasan perlangkah soal02 sebagai berikut:

1. def deret(n): -> untuk mendefinisikan fungsi bernama def deret(n).
2. for i in range(n, 0, -1): -> perulangan luar yang akan berjalan dari n sampai 1 (mundur). Setiap iterasi mewakili satu baris output.
3. faktorial = 1 -> Menginisialisasi variabel faktorial dengan nilai 1 sebelum menghitung faktorial dari i. Reset setiap kali memulai perulangan i baru.
4. for j in range(i, 0, -1): -> Perulangan dalam yang akan mengalikan bilangan dari i sampai 1 untuk menghitung faktorial.
5. faktorial = faktorial * j -> Menghitung faktorial dengan mengalikan faktorial dengan j secara berulang.
6. print(f"{faktorial} ", end="") -> Mencetak nilai faktorial yang sudah dihitung, diikuti spasi, dan tidak pindah baris (agar output dalam satu baris).
7. for k in range(i, 0, -1): -> Perulangan untuk mencetak deret bilangan dari i sampai 1 (mundur).
8. print(f"{k}", end=" ") -> Mencetak nilai k (bilangan dari i sampai 1) diikuti spasi, dan tidak pindah baris.
9. print() -> Setelah selesai mencetak satu baris (faktorial dan deret bilangan), program akan pindah ke baris baru untuk baris berikutnya.

10. n = int(input("n = ")) -> Meminta pengguna memasukkan nilai n dan mengkonversinya ke integer.
11. deret(n) -> Menjalankan fungsi deret yang sudah didefinisikan dengan nilai n yang diinput pengguna.

SOAL 3

PrakAlPro-71251218 > minggu07 > soal03.py > ...

```python
def deret(tinggi, lebar):
    angka = 1
    for i in range(tinggi):
        for j in range(lebar):
            print(f"{angka} ", end="")
            angka = angka + 1
        print()
tinggi = int(input("tinggi = "))
lebar = int(input("lebar = "))
deret(tinggi, lebar)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> py
tinggi = 6
lebar = 7
1 2 3 4 5 6 7
8 9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31 32 33 34 35
36 37 38 39 40 41 42
```

Penjelasan perlangkah soal03 sebagai berikut:

1. def deret(tinggi, lebar): -> untuk mendefinisikan fungsi bernama deret(tinggi, lebar).
2. angka = 1 -> menginisialisasi variabel angka dengan nilai 1. Angka ini akan terus bertambah setiap kali dicetak.
3. for i in range(tinggi): -> Perulangan luar yang mengontrol jumlah baris. Setiap iterasi i mewakili satu baris baru dalam matriks.
4. for j in range(lebar): -> Perulangan dalam yang mengontrol jumlah kolom. Perulangan ini berada di dalam perulangan i, artinya untuk setiap baris, akan dicetak sebanyak lebar angka.
5. print(f"{angka} ", end="") -> Mencetak nilai angka diikuti spasi, dan tidak pindah baris (karena end=""). Hal ini membuat semua angka dalam satu baris tetap tercetak berurutan.
6. angka = angka + 1 -> Setelah mencetak angka, nilai angka ditambah 1. Ini memastikan angka berikutnya yang dicetak adalah angka yang berurutan.
7. print() -> Setelah selesai mencetak satu baris penuh (setelah perulangan j selesai), program akan pindah ke baris baru untuk mencetak baris berikutnya.

8. tinggi = int(input("tinggi = ")) -> Meminta pengguna memasukkan nilai tinggi (jumlah baris) dan mengkonversinya ke integer.
9. lebar = int(input("lebar = ")) -> Meminta pengguna memasukkan nilai lebar (jumlah kolom) dan mengkonversinya ke integer.
10. deret(tinggi, lebar) -> Menjalankan fungsi deret yang sudah didefinisikan dengan nilai tinggi dan lebar yang diinput pengguna.